University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Final Examination, November 2014

Read through the entire examination paper before you start writing so that you can answer the easy questions first. The relative importance of a question is indicated by the marks allocated to it. Please write legibly.

This paper consists of Four Sections:

Section 1: Software Design — 3 compulsory questions

Section 2: Compilers — 2 compulsory questions

Section 3: Theory of Algorithms — 2 compulsory questions

Section 4: Choice Section — Choice of 3 out of 6 questions.

You must answer **all** the questions for each of Sections 1–3, and answer **three questions** from the six in the Choice Section (Section 4).

Total Marks: 90	Approximate marks per question are shown in brackets.
Time: 3 hours + 10 minutes reading	The use of calculators is permitted.

Your attention is drawn to the instructions on your answer book on the front and the back. In particular "**EVERY CANDIDATE MUST enter the book number and the number of each question answered (in the order in which it has been answered)**" on the table provided on the front cover of the first book.

Section 1. Advanced Software Design [22 Marks]

Question 1. **[5]**

a. Why is the "critical path" in a network analysis of tasks a manager's main concern? What does it tell him/her? [2]

b. How do you deal with risks in the course of a project once you have identified and analysed them? [3]

Question 2. **[7]**

Discuss how Scrum encourages a sense of commitment and ownership in the team.

Question 3. **[10]**

a. A basic architectural design principle is to separate concerns and localize impact. Explain with an example what is meant by this and what the benefits are of doing this. [3]

A programming language compiler can include the following components:

1) A lexical analyser
2) A symbol table, which holds information about the names of entities used in the text that is being translated.
3) A syntax analyser.
4) A syntax tree, which is an internal structure representing the program being compiled.
5) A semantic analyser.
6) A code generator.

b. Choose and architectural pattern for a compiler, name it, and draw a diagram of such a compiler architecture labelling all the parts. [5]

c. You believe that people should own their own software, code and all, and companies shouldn't have the right to grab previously open software in packages and so force people to buy products that started out as truly free. What type of open source license would you want to use? Justify your choice. [2]

Section 2. Compilers [22 Marks]

Question 4. **[13]**

a. Consider the programming languages Python, Java and C++. Which one is the odd one out, considering the format these languages are translated to, prior to execution? Explain your choice. [2]

Consider the context-free grammar **E → E E | (E) | E + E | E * | a** and the string **(a + a) * a** and answer the questions below:

b. Give a leftmost derivation for the string. [3]

c. Give a parse tree for the string. [3]

d. Consider the following Java program which each contains 2 unique errors. Indicate which line the error is on as identified on the left, what the error is and also indicate which phase of the compilation process the error is identified - Lexical Analysis, Parsing(Syntactic Analysis) or Semantic Analysis. [4]

```
1. import java.util.*;
2.
3. class SumPrice
4. {
5.    public static void main(String[] args)
6.    {
7.       float sum = 0;
8.       int items;
9.       char answer;
10.      Scanner kb = new Scanner(System.in);
11.      do
12.      {
13.         System.out.print("Enter item price:");
14.         price = kb.nextFloat();
15.         System.out.print("Enter number of items:");
16.         items = kb.nextInt();
17.         sum += price * items;
18.         System.out.print('Done(y/n)?:');
19.         answer = kb.next().charAt(0);
20.      }
21.      while(answer == 'n' || answer == 'N');
22.      System.out.print("Sum price is " + price);
23.   }
24.}
```

e. Specify the purpose of an activation record. [1]

Question 5. **[9]**

Consider the following program, ready for code block reorganisation:

Start: Statement0

A: Statement1

Jump C

B: Statement2

Jump D

C: Statement3

Jump B

D: Statement4

Done:

a. Rearrange the blocks into traces and then show how the code would be optimised by drawing a line through redundant jumps. [3]

Consider the following list of Canonicalization rules:

```
ESEQ(s1, ESEQ(s2, e)) => ESEQ(SEQ(s1,s2), e)

BINOP(op, ESEQ(s,e1),e2) => ESEQ(s, BINOP(op, e1, e2))

MEM(ESEQ(s, e1)) => ESEQ(s, MEM(e1))

JUMP(ESEQ(s, e1)) => SEQ(s, JUMP(e1))

MOVE(ESEQ(s, e1), e2) => SEQ(s, MOVE(e1, e2))

MOVE(e1, ESEQ(s, e2)) => SEQ(MOVE(TEMP t, e1), SEQ(s, MOVE(TEMP t, e2)))
```

b. Transform this IR tree as much as possible, using the above rules. Show each rule being applied as a separate step. [3]

```
MOVE(MEM(BINOP(op,ESEQ(s,e1),e2)),e3)
```

c. What is the primary data structure used during Liveness Analysis? [1]

d. Compiler best practices have evolved over time. Indicate two practices/policies which have existed in older compilers, but typically no longer exist in modern compilers. [2]

Section 3. Theory of Algorithms [22 Marks]

Question 6. Basic knowledge [12]

a. Briefly (one sentence for each) explain the main purpose of gprof and the main purpose of valgrind. [2]

b. It has been proven that sorting is Ω (n log n). What does this mean? [1]

c. You were shown a distribution sorting algorithm that is Θ(n). Explain why this does not contradict your answer in b? [1]

d. In a GPS application, to find the fewest roads between two places, would you use a breadth-first or depth-first search? Briefly justify your answer (i.e. one short sentence). [2]

e. You need to read a file of strings into an array. Describe two algorithms (one short sentence for each) to check the array to see if it has any duplicates. Also give the asymptotic efficiency class for each algorithm. [3]

f. Given a text and a pattern P[0..m-1], complete writing these two rules for creating the shift table, T, for the Horspool string searching algorithm. [2]

T(c) = … for a letter, c, in P [0..m-2]

T(c) = … for a letter not in P[0..m-2]

g. Explain the difference between P and NP. [1]

Question 7. Applying knowledge [10]

Consider the pseudocode for these two divide and conquer algorithms for multiplying two n x n matrices, A and B.

Algorithm 1.

```
1. MM(A, B):
2.     Split matrices into 4 n/2 X n/2 parts A11, B11, … , A22, B22
3.     If n == 1:
4.         Return A11 * B11
5.     Else:
6.         Compute MM (A11 , B11 ) + MM(A12 , B21 )
7.         Compute MM (A11 , B12 ) + MM (A12 , B22 )
8.         Compute MM (A21 , B11 ) + MM (A22 , B21 )
9.         Compute MM (A21 , B12 ) + MM (A22 , B22 )
```

Algorithm 2.

```
1. MM(A, B):
2.     If n == 1:
3.         Return A11 × B11
4.     Else:
5.         Split matrices into 8 n/2 X n/2 parts: A11 , B11 , . . . , A22 , B22
6.         P1 = MM(A11, B12 − B22 )
7.         P2 = MM(A11 + A12 , B22 )
8.         P3 = MM(A21 + A22 , B11 )
9.         P4 = MM(A22 , B 21 − B11 )
10.        P5 = MM(A11 + A22 , B11 + B22 )
11.        P6 = MM(A12 − A22 , B21 + B22 )
12.        P7 = MM(A11 − A21 , B11 + B12 )
13.        C11 = P5 + P4 − P2 + P6
14.        C12 = P1 + P2
15.        C21 = P3 + P4
16.        C22 = P1 + P5 − P3 − P7
17.        Return  C
```

a. One of the algorithms has the same asymptotic efficiency class as brute force matrix multiplication. Which one? [1]

b. What is the name of the other algorithm? [1]

c. Write a recurrence relation for algorithm 2 and show using the master theorem what its asymptotic efficiency class is. [2]

d. You have to implement a matrix multiplication algorithm for an application in which the matrices are never more than 1000 x 1000. Would you use the brute force algorithm we did in class, algorithm 1 or algorithm 2? Justify your answer in one brief sentence. [1]

e. Matrix multiplication is slow but algorithms to do it are polynomially bounded. Identify one of the several problems studied in class for which there are no polynomially bounded algorithms. [1]

f. Identify, and then describe in one sentence an example of an NP-complete problem. [2]

g. A fellow-student understands properly what a Turing Machine is, but does not understand what is meant by a *non-deterministic* one. Explain it to him or her in no more than 45 words. [2]

Section 4. Choice Section: Answer any THREE of the following six questions. [24]

Question 8. ASD 1: Distributed Systems Architecture [8]

Describe a general broker architecture for distributed systems and provide a diagram of a simplified broker architecture. Discuss the advantages of a general broker architecture over client server architecture in distributed system design.

Question 9. ASD 2: Safety-Critical Projects [8]

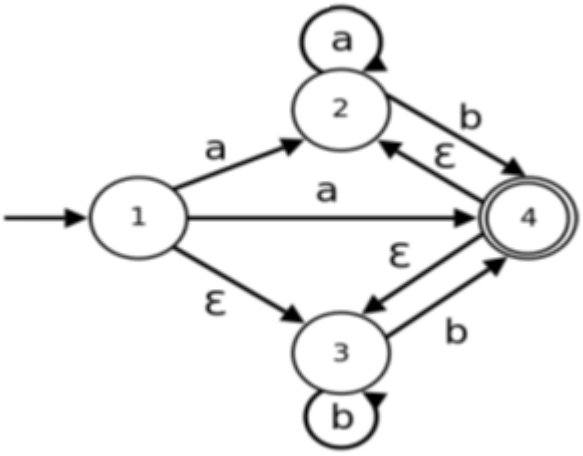
A safety-critical project is one where safety is of utmost importance and where faults or failure may result in injury, loss of life or serious damage to the environment or property. In short the consequences of failure are unacceptable. Many organizations, however, are hesitant to employ agile methods for their safety-critical systems. They believe there are serious concerns about using such methods and therefore compromising those systems.

Discuss whether the waterfall model of software development is more, or less, suitable for safety-critical systems than agile methods. Justify your answer.

Question 10. Compilers - Front End [8]

a. When defining Regular Expressions, explain what the Kleene closure operation does. [1]

b. Convert the following Nondeterministic Finite Automaton (NFA) to a Deterministic Finite Automaton (DFA). [6]



c. Which compilation stage uses a Symbol Table? [1]

Question 11. Compilers - Back End [8]

a. Rewrite the following Java code fragment below to a more optimal form using constant folding and loop strength reduction. [2]

```
for(int a=0; a<4; a++) {
    System.out.println(3 * 4 * a);
}
```

Consider the following live-in/out sets yielded by Liveness analysis: {ax} {xb} {cy}

Assume that x and y are precoloured nodes. Also assume that a and y are move related, and y and b are move related.

b. Based on just this information, draw a corresponding interference graph. [3]

c. Apply the heuristic graph colouring register allocation algorithm (simplify, coalesce, freeze, spill, select) to the interference graph. Assume K=2, with the registers named R1 and R2. Show each step in the algorithm separately, and indicate the final allocation of registers to variables. [3]

Question 12. ToA – Efficient closest points [8]

Consider the following pseudo-code to find the shortest distance between a set of points on a Euclidean plane, each with an x and y co-ordinate.

1. Sort the points by x order (tiebreak on the y value) into P.
2. Sort the points by y order (tiebreak on the x value) into Q.
3. EfficientClosestPair(P, Q)

4. EfficientClosestPair(P, Q):
5. if n <= 3: return minimal distance found by brute force
6. copy the first ceiling[n/2] points of P to Pl
7. copy the same points of Q to Ql
8. copy the remaining floor[n/2] points of P to Pr
9. copy the same points of Q to Qr
10. dl = EfficientClosestPair(Pl, Ql)
11. dr = EfficientClosestPair(Pr, Qr)
12. d = min(dl, dr)
13. m = P[ceiling(n/2)]-1].x
14. copy all points of Q for which |x-m|<d into S[0..num-1]
15. dminsq = d * d
16. for i = 0 to num – 2:
17. k = ….
18. while k <= num – 1 and (S[k].y – S[i].y)^2 < dminsq:
19. dminsq = min((S[k].x-S[i].x)^2+(S[k].y-S[i].y)^2, dminsq)
20. k = k + 1
21. return sqrt(dminsq)

a. Complete line 17. [1]

b. Lines 16 to 20 appear to be $\Theta(n^2)$ in the worst case. Explain why this isn't true, and what the worst case efficiency actually is for these lines of code. [3]

c. Superficially, line 7 also appears to be $\Theta(n^2)$. Use pseudocode to show how this can be done with a $\Theta(n)$ algorithm. [4]

Question 13. ToA - Partitioning [8]

Consider the following code:

```
1.  Partition(A[l .. r]):
2.          p = A[l]
3.          i = l
4.          j = r + 1
5.          repeat:
6.                  repeat ++i until A[ i ]>= p
7.                  repeat --j until A[ j ]<= p
8.                  swap(A[ i ], A[ j ])
9.          until i >= j
10.         swap(A[ i ], A[ j ])
11.         swap(A[ l ], A[ j ])
12.         return j
```

a. This is one of the two partitioning algorithms learned in class. Write down the name of this one. Then, after it, write down the name of the other one. [1]

b. To find the k*th* smallest element in an array you can either use Quickselect or you can sort the array and then take the k*th* element. Of these two algorithms you are told to implement the one that does better on average. Which one would you implement and why? In answering, explain the worst and average case asymptotic efficiency class for both algorithms. [2]

c. For Quickselect and the standard implementation of Quicksort, this partitioning algorithm has a bug. Explain the bug briefly. [2]

d. How could we change the way the pivot is selected to remove this bug. [1]

e. Identify two additional ways (i.e. not the method used in the above question) to reduce the risk of Quicksort degrading to $O(n^2)$ behaviour. [2]